



Clase 3 – Construir un Agente con IA

No-Code vs. Código Asistido

Programa de Especialización IA Generativa y LLMs para la Transformación Organizacional · FEN · Abril 2026

Estructura de la sesión



Cátedra

Arquitectura, opciones y tradeoffs



Demo en vivo

v0.app + Claude Code



Taller

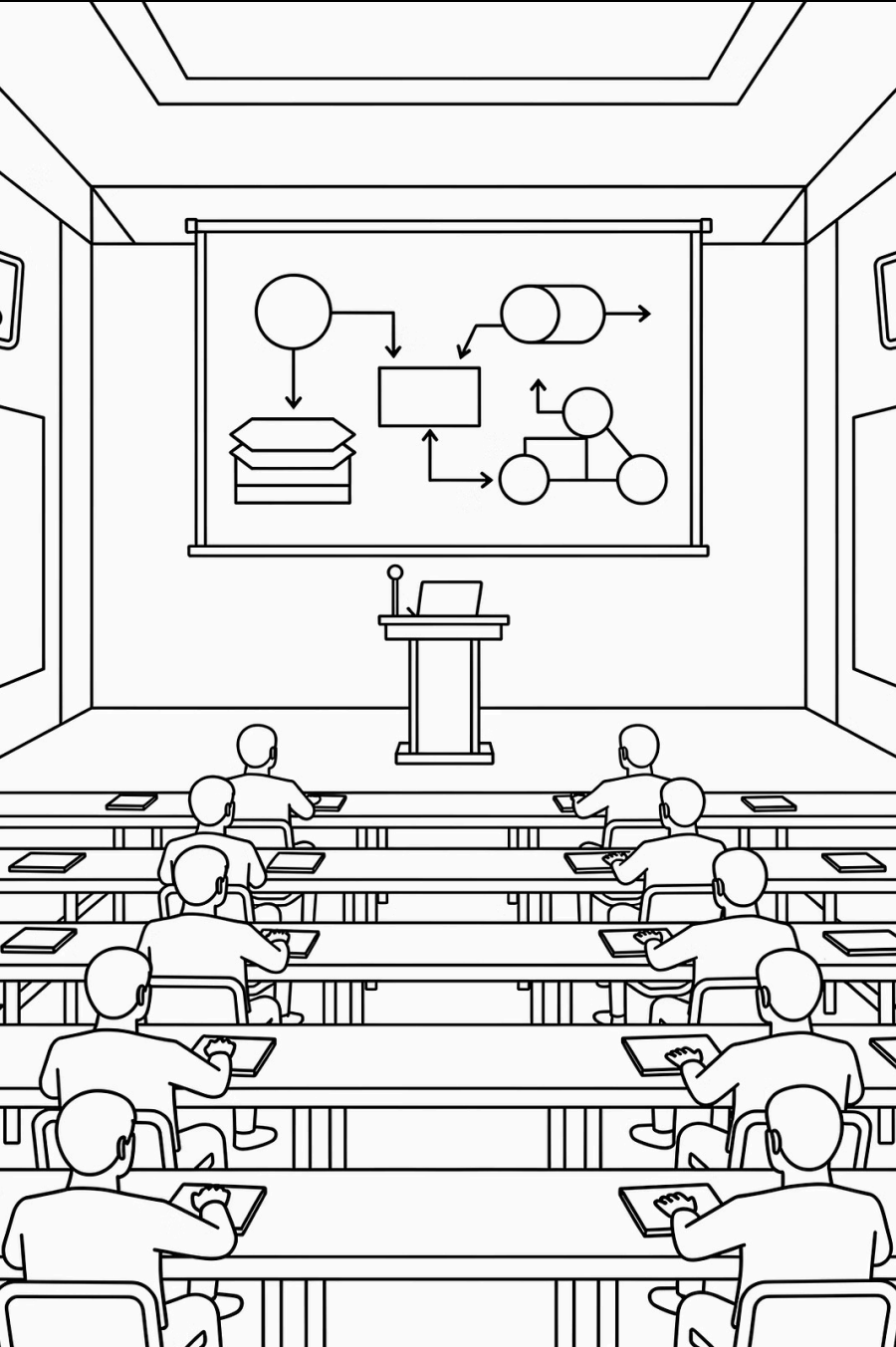
Construir el agente



Cierre

¿Cuándo usar cada una?

 Hoy salimos con un agente funcional construido por ustedes.



SESIÓN DE HOY

Bloque 1 · Cátedra

40 minutos de fundamentos, arquitectura y decisiones de diseño antes de construir.

Recapitulación rápida

Las tres clases forman un sistema coherente. Cada una construye sobre la anterior.

1

Clase 1 – El prompt como contrato

Estructura, temperatura y límites.
Comunicarse con precisión con un LLM.

2

Clase 2 – Agentes que perciben y actúan

Cómo los agentes razonan y usan herramientas para completar tareas.

3

Clase 3 – Construir de principio a fin

Dos caminos distintos, el mismo destino: un agente real y funcional.

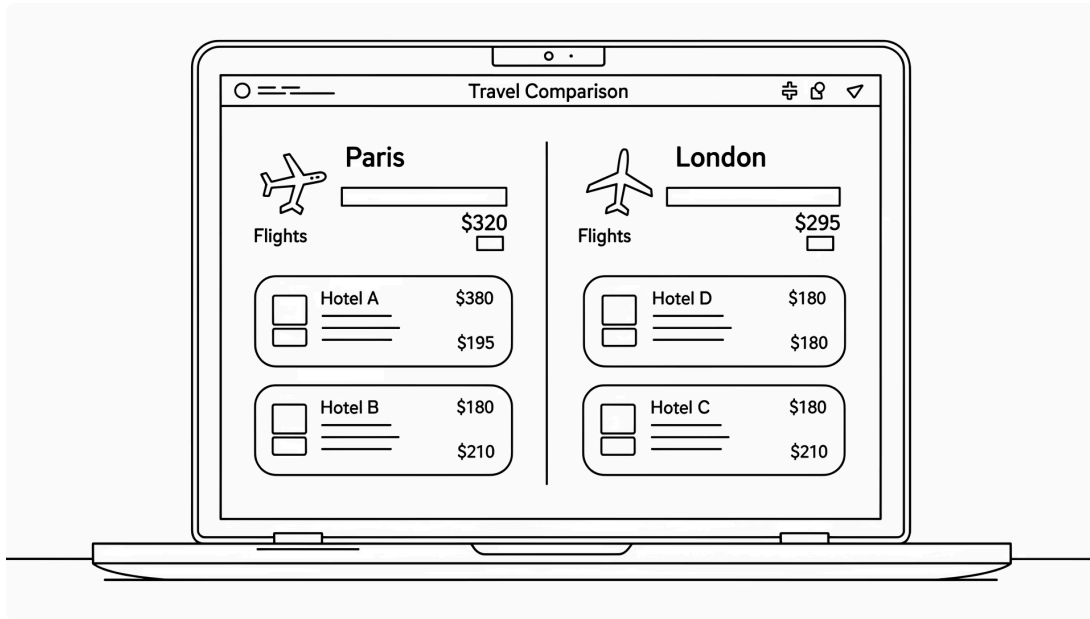
 El agente de hoy compara dos destinos de viaje, genera un PDF y lo envía por correo.

El agente que vamos a construir

Agente Comparador de Viajes

El usuario escribe: **"Compara París vs Londres, saliendo desde Santiago"**

El agente ejecuta cinco pasos en secuencia y entrega un PDF comparativo enviado por correo.



01

Busca vuelos

02

Busca hoteles

03

Busca atracciones

04

Calcula presupuesto

05

Compara y recomienda

Las dos vías

No-code vs. código asistido por IA: ambas son válidas. La clave está en saber cuándo usar cada una.

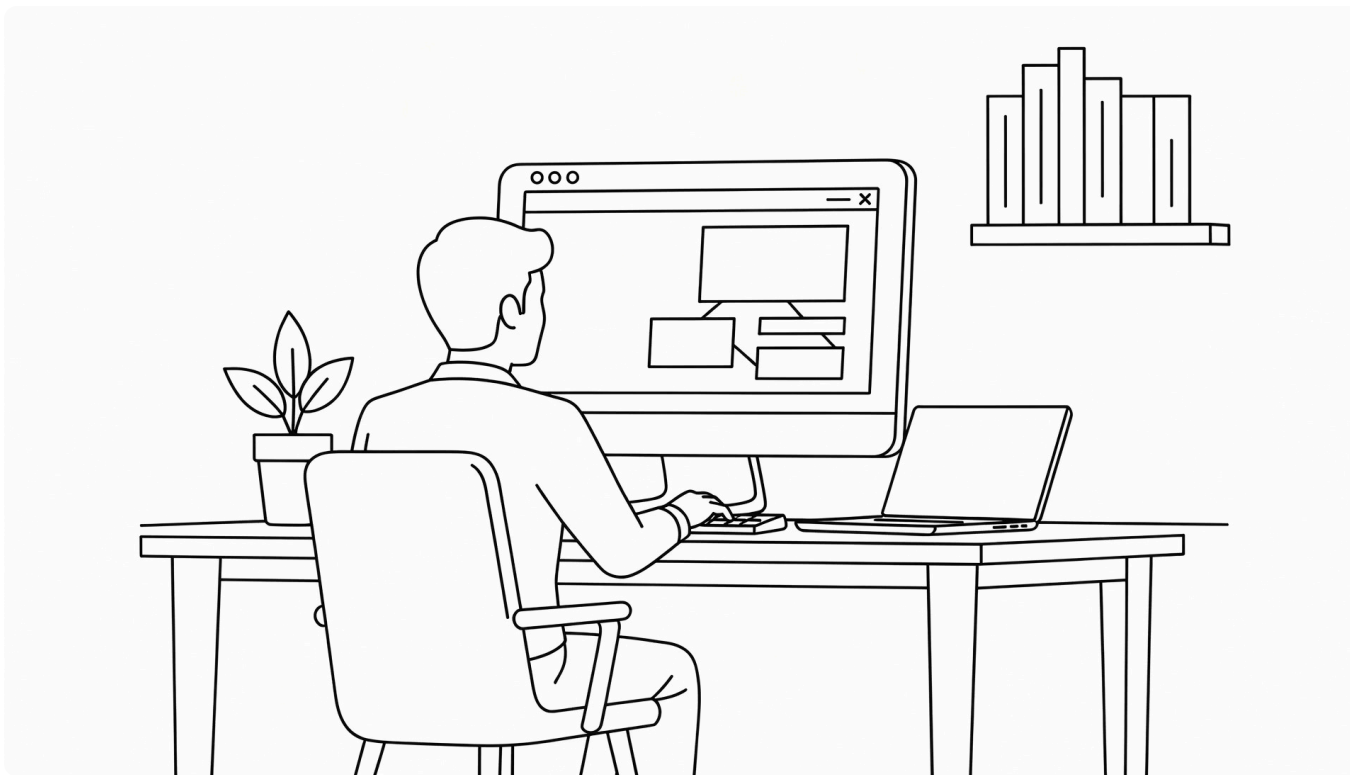
v0.app · No-Code

- Usuarios: negocio, analistas, diseñadores
- Prototipo en minutos
- Curva de aprendizaje muy baja
- Flexibilidad limitada al frontend generado
- Plan gratuito con límites

Claude Code · Código Asistido

- Usuarios: developers y analistas técnicos
- Prototipo en horas
- Curva de aprendizaje media
- Flexibilidad total sobre el proyecto
- Costo por token de API

📌 **Ambas sirven.** La pregunta es: ¿para qué y cuándo?



VÍA NO-CODE

v0.app: qué es

Describes en texto lo que quieres y genera el código de inmediato. Sin instalación, sin configuración — todo en el navegador.

- 📄 El **tradeoff** central: velocidad y facilidad a cambio de control sobre el código generado.

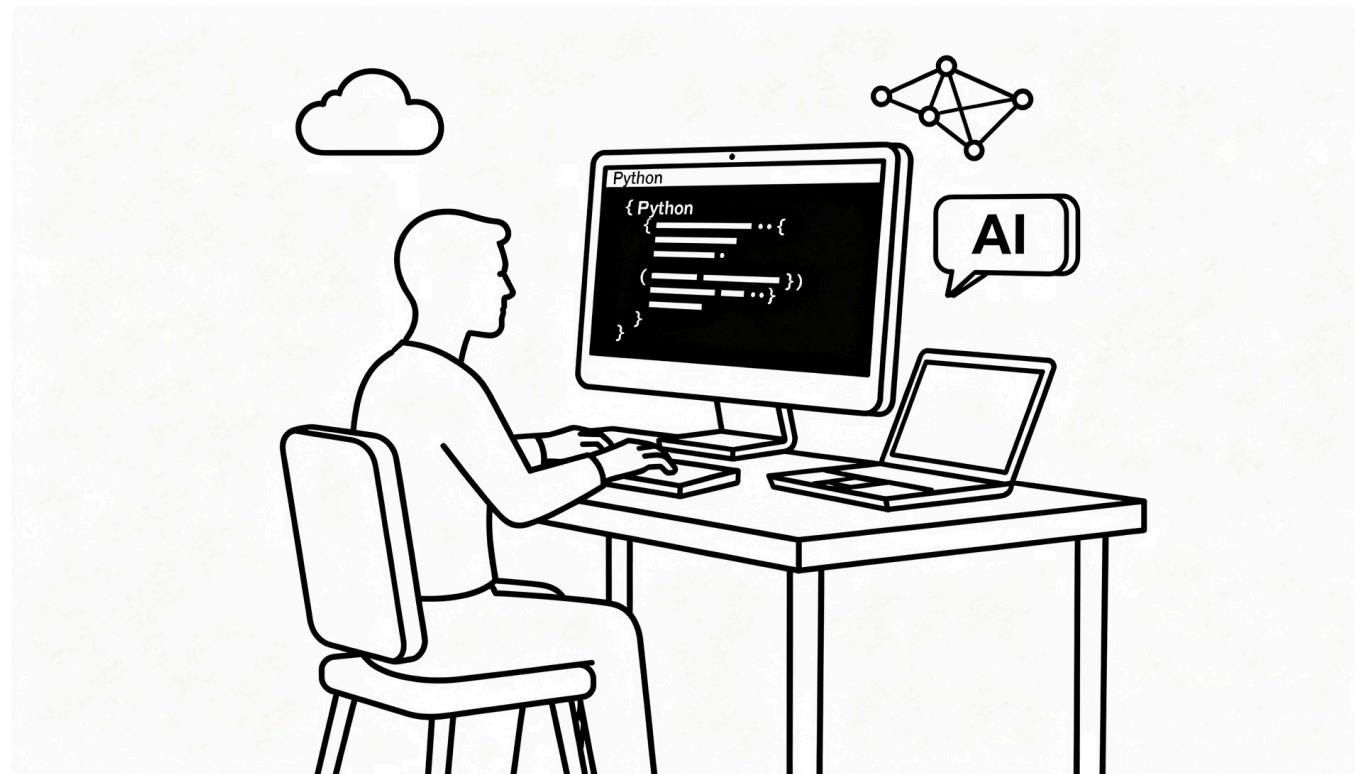
- Interfaces funcionales en React o HTML puro
- Ideal para prototipar rápido y validar diseños
- Equipos sin experiencia en frontend

VÍA CON CÓDIGO

Claude Code: qué es

Un asistente de terminal que entiende tu proyecto completo. Escribe código, lo ejecuta, detecta errores y los corrige — con archivos reales.

📄 **El tradeoff central:** más control, más responsabilidad.



→ **Trabaja con tu proyecto completo, no abstracciones visuales**

→ **Ideal para proyectos que van a producción**

→ **Integración con sistemas y lógica compleja**



El prompt importa más de lo que parece

✗ Prompt vago

"Compara viajes"

El agente no sabe qué comparar, qué datos buscar, cómo formatear el resultado ni cuándo detenerse.

✓ Prompt estructurado

"Eres un agente comparador de viajes. Recibes dos destinos y un origen. Tu tarea es: obtener precios reales, calcular presupuesto total por 7 días, identificar mejor relación precio-valor y generar un informe estructurado. Usa solo precios verificables."

📌 El prompt define el comportamiento, los límites y la calidad del output.

Anatomía de un buen prompt de agente

Las cinco partes que no pueden faltar. Sin límites explícitos, el LLM improvisa — y eso en negocios es un problema.

1

Rol

"Eres un agente comparador de viajes"

2

Contexto

"Datos de mercado 2026, cliente chileno"

3

Tarea

"Compara dos destinos en presupuesto y valor"

4

Formato

"Tabla + recomendación + justificación"

5

Límites

"Solo precios verificables. Sin inventar datos."

Agentes: qué ayudan y qué no reemplazan

La honestidad que falta en el mercado.

✓ Lo que hacen bien

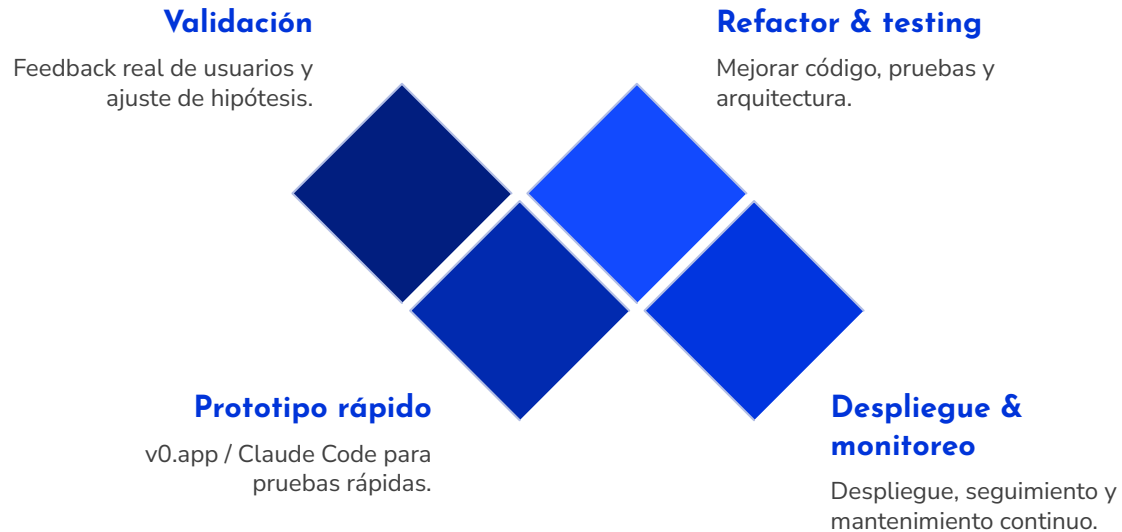
- Prototipos en horas, no semanas
- Automatizar tareas repetitivas y bien definidas
- Integrar sistemas antes complejos
- Validar ideas antes de invertir en ingeniería

✗ Lo que NO reemplazan

- Diseño de arquitectura de sistemas
- Testing, seguridad y manejo de errores
- Privacidad y compliance
- Mantenimiento a largo plazo
- Lógica de negocio compleja y auditada

- 📋 Un agente hecho con IA puede darte un prototipo funcional en un día. Convertirlo en producto de producción sigue requiriendo ingeniería de software.

El ciclo real de un proyecto con agentes



La IA acelera las primeras etapas. **No elimina las últimas.**

¿En qué etapa estamos hoy?

En el **prototipo rápido**: la etapa más visible y emocionante del ciclo.

Las etapas siguientes —validación, refactor, deploy— **no son opcionales para proyectos reales**. Son el trabajo de ingeniería que convierte una demo en un producto.

- 📌 Saber dónde estás en el ciclo te ayuda a tomar mejores decisiones sobre qué herramienta usar.

Qué pasa cuando vas directo a producción sin ingeniería

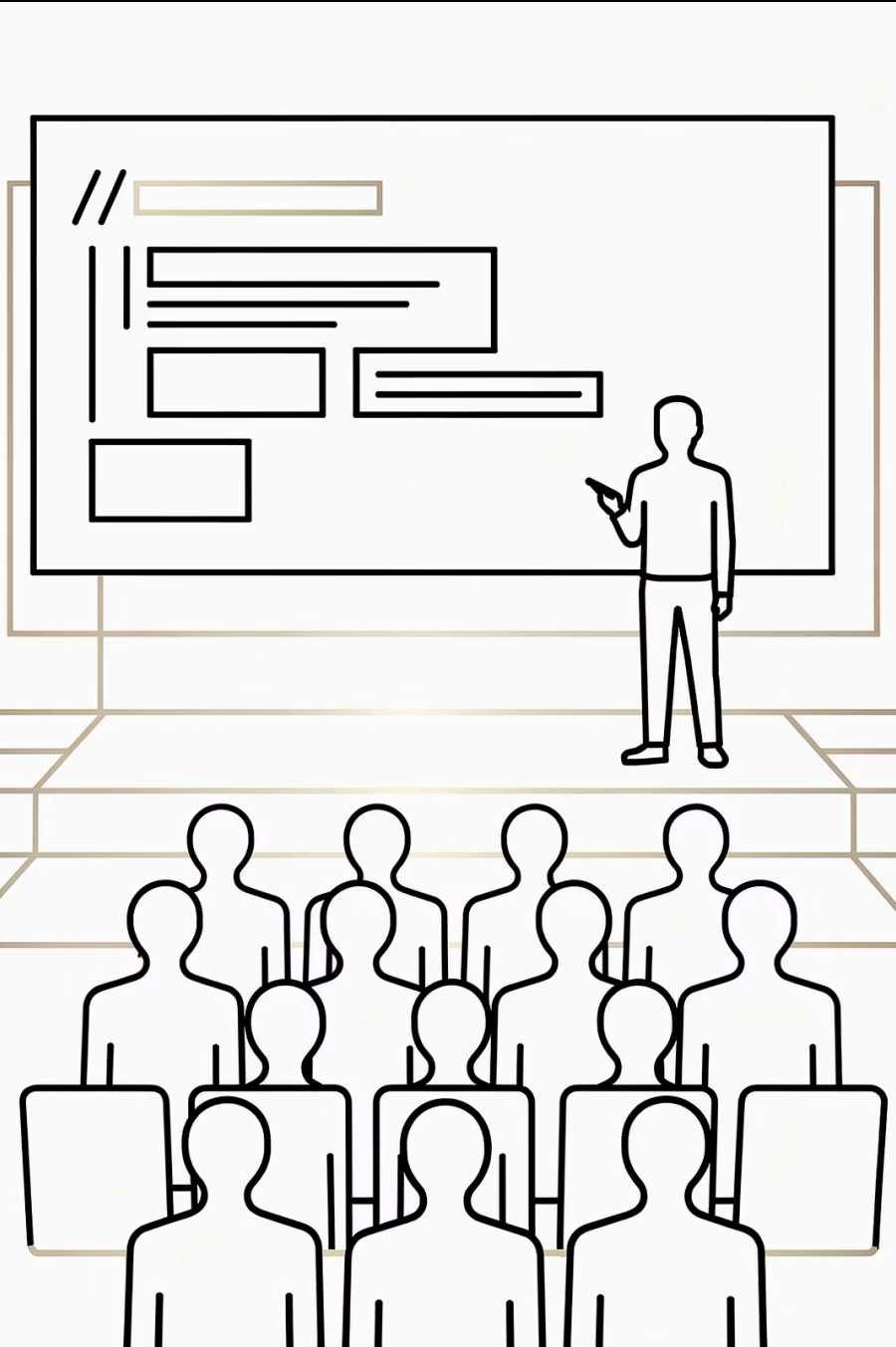
Problema	Causa raíz
El agente "alucina" precios	Sin validación de datos
Falla silenciosamente	Sin manejo de errores
No escala	Sin arquitectura de carga
Imposible auditar	Sin logs ni trazabilidad
Datos personales expuestos	Sin revisión de seguridad
El equipo no puede mantenerlo	Sin documentación ni tests

❏ El prototipo impresiona. La producción exige.



Break · 10 minutos

Regresamos con la demo en vivo.



SESIÓN DE HOY

Bloque 2 · Demo en Vivo

Construimos el agente en tiempo real, dos caminos distintos.

Demo 1: v0.app

Construir la interfaz del agente con un prompt

El prompt que vamos a usar

Una página única "Comparador de Viajes" con formulario de origen/destinos/fecha/correo, botón de acción y visualización de resultados. Diseño minimalista en blanco y negro, ancho máximo 440px, sin frameworks.

- Un solo archivo HTML con CSS y JavaScript "vainilla" — sin frameworks ni compilación.

Lo que vamos a observar juntos

- ¿Qué interpretó bien el modelo?
- ¿Qué cambió sin pedírselo?
- ¿Podemos conectarlo al backend real?

Demo 2: Claude Code

El mismo agente, desde el terminal

Estructura del proyecto

```
agente-viajes/  
├── main.py      # Orquestador  
├── scraper/  
│   ├── flights.py # Precios de vuelos  
│   ├── hotels.py  # Precios de hoteles  
│   └── attractions.py  
└── report/  
    ├── pdf_generator.py  
    └── email_sender.py # Resend
```

Ejecutar: `uv run python main.py --opcion-a Paris --opcion-b Londres`

Lo que vamos a observar

El PDF generado

Informe comparativo completo

El correo enviado

Entregable automático vía Resend

El código auditable

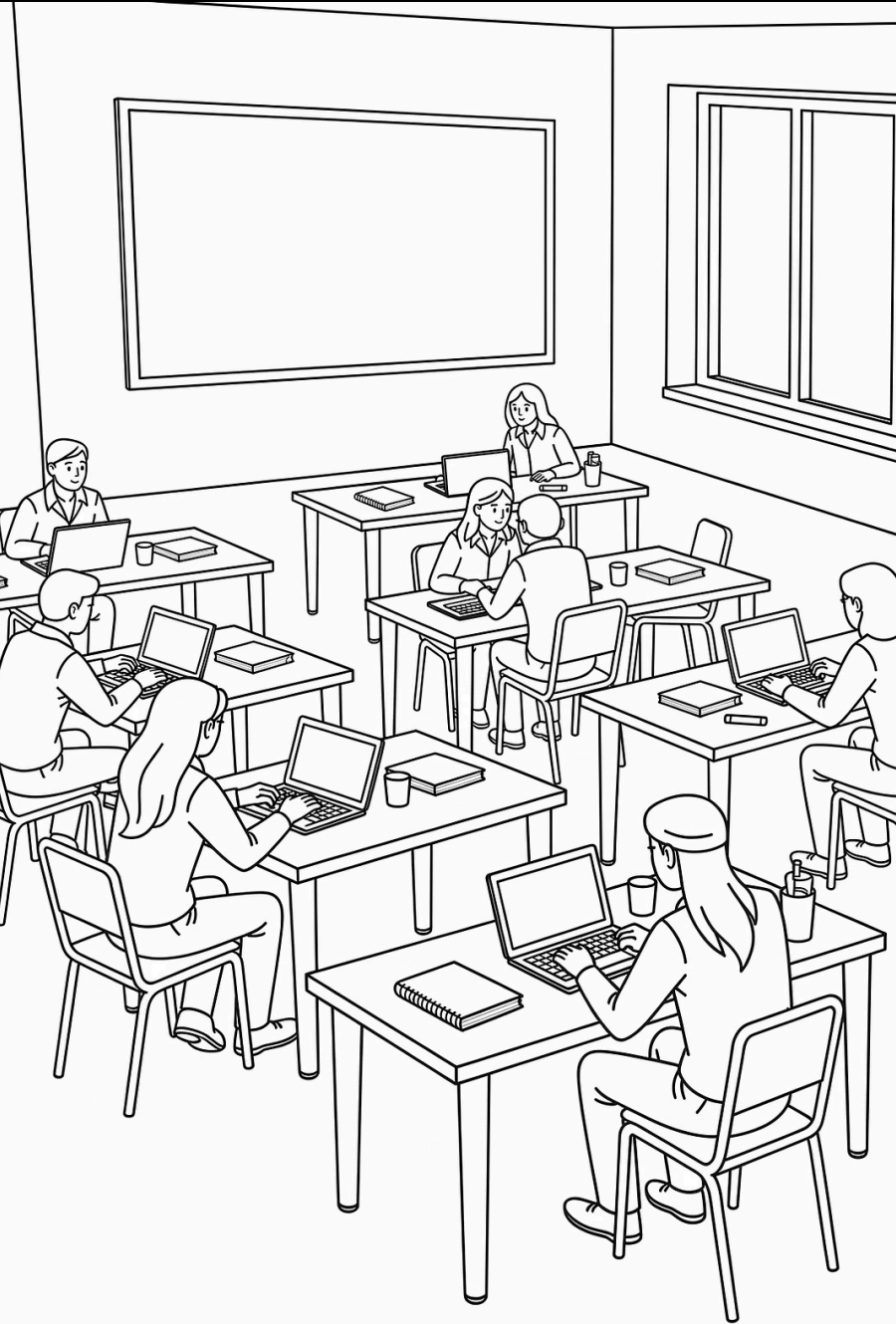
Cada paso visible y modificable

Comparación post-demo

Lo que acabamos de ver en perspectiva.

Aspecto	v0.app	Claude Code
Modificar comportamiento	Nuevo prompt o editar UI	Editar directamente el archivo
Conectar al backend	Manual, copiar y pegar	Integrado desde el inicio
Ver qué salió mal	Solo en el navegador	Logs del terminal + código
Auditar cada cálculo	No (solo frontend)	Sí, completamente

❏ Ninguna vía es mejor en absoluto — depende del contexto, el equipo y el objetivo.



SESIÓN DE HOY

Bloque 3 · Taller

60 minutos · Su turno de construir.

Instrucciones del taller

Grupos de 3 personas. Eligen una vía. Entregable: 3 minutos de presentación + reflexión.

Vía A – v0.app

1. Ir a v0.app (no requiere instalación)
2. Pegar el prompt de la diapositiva 11
3. Ajustar diseño con prompts adicionales
4. Documentar: ¿qué interpretó bien? ¿qué tuvo que corregir?

Vía B – Claude Code

1. Clonar o explorar agente-viajes/
2. Añadir una nueva comparación (ej: costo de visa)
3. Ejecutar y verificar el output
4. Documentar: ¿qué cambió? ¿qué fue difícil?

Guía de prompts para el taller

Prompts listos para usar con Claude Code durante el taller.

Para explorar el código

"Explícame qué hace cada módulo en este proyecto y cómo se conectan entre sí"

Para añadir funcionalidad

"Añade al agente la capacidad de calcular el costo estimado de visa para un chileno viajando a [destino]. Usa datos estáticos si no hay API disponible."

Para entender un error

"Este error ocurre cuando ejecuto main.py: [pegar error]. ¿Qué lo causa y cómo lo corrijo?"

Para mejorar el prompt del agente

"El agente no explica por qué recomienda un destino. Modifica el prompt para que justifique con al menos 3 criterios."

SESIÓN DE HOY

Bloque 4 · Cierre

10 minutos · Reflexión final y síntesis.



¿Cuándo usar cada vía?

Marco de decisión para elegir la herramienta correcta según el contexto.

Contexto	Vía recomendada
Quiero validar una idea en un día	v0.app
El proceso procesará datos sensibles	Código (auditabilidad)
El equipo no tiene desarrolladores	v0.app
Necesito integrarme con sistemas legados	Código
Va a operar sin supervisión humana	Código + tests
Quiero que el equipo lo mantenga en 2 años	Código
Necesito escalar a miles de usuarios	Código

☐ La plataforma no-code es excelente para descubrir. El código es necesario para construir.

El prompting no es opcional

Lo aprendimos construyendo

Con un prompt vago

El agente inventa datos

No sabe cuándo detenerse

No maneja errores

Output inconsistente

Con un prompt bien estructurado

Comportamiento
predecible

Output en el formato
esperado

Manejo explícito de casos
borde

Más fácil de depurar y
mejorar

📌 El prompt es el código de los agentes no-code. Si no lo cuidas, el agente falla.

Lo que los agentes IA no resuelven por sí solos:



Seguridad y privacidad

Autenticación, autorización, cumplimiento normativo y protección de datos personales siguen siendo responsabilidad del equipo de ingeniería.



Calidad y observabilidad

Testing automatizado, manejo de errores, reintentos, logs y alertas no se generan solos — requieren decisiones técnicas explícitas.

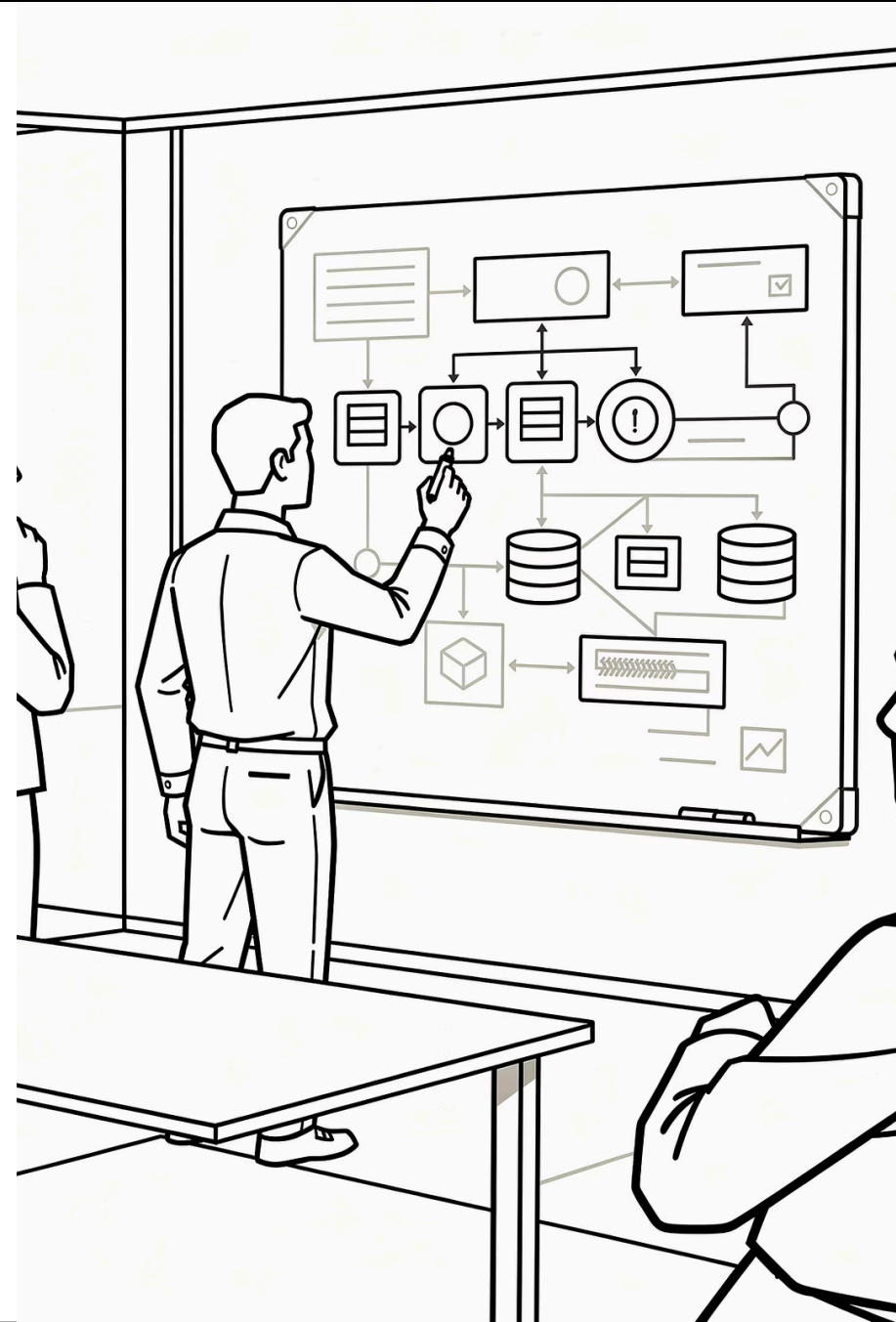


Mantenimiento a largo plazo

Gestión de dependencias, actualizaciones, documentación y performance bajo carga son trabajo de ingeniería, independiente de cómo se generó el código.



La IA puede escribir el código. Pero alguien tiene que entenderlo, revisarlo y mantenerlo.



El nuevo rol del profesional

Lo que cambia y lo que no

Lo que cambia

- Puedes prototipar sin depender 100% de un desarrollador
- Puedes automatizar tareas personales que antes tardaban horas
- Puedes iterar sobre ideas mucho más rápido

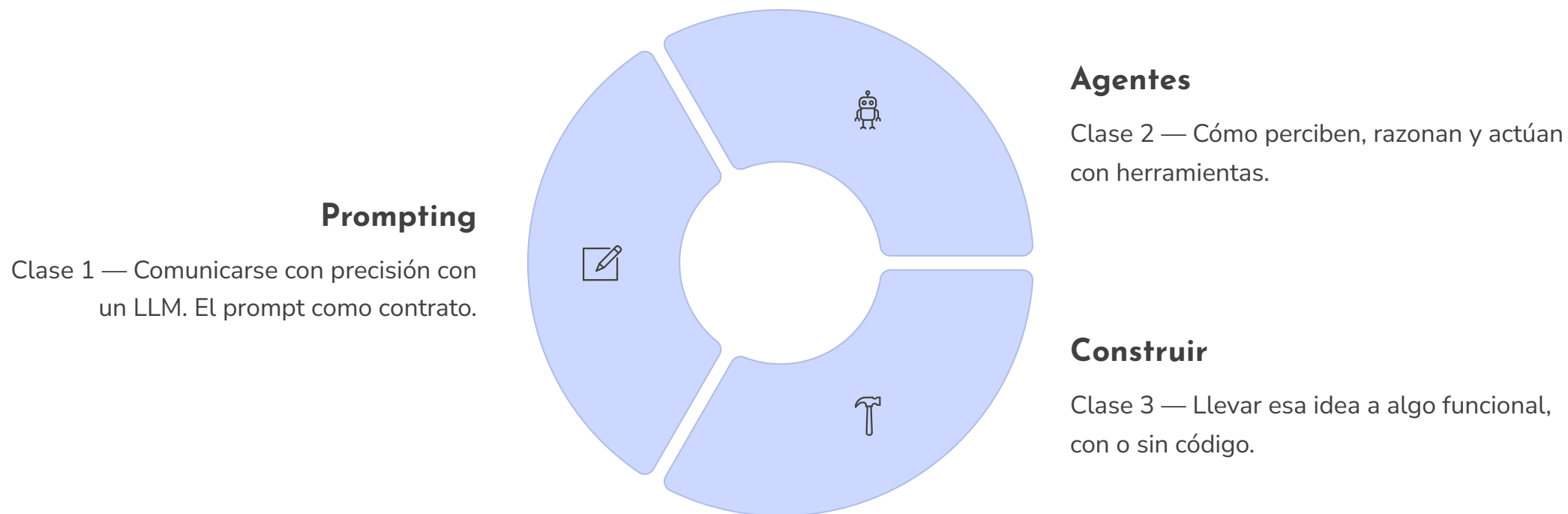
Lo que NO cambia

- Necesitas entender el problema de negocio
- Necesitas saber si el output tiene sentido
- Eres responsable del resultado, aunque lo generó la IA

 La IA Generativa y la IA Agéntica amplifican tu capacidad. No reemplazan tu criterio.

Síntesis de las 3 clases

Los tres ejes del diplomado forman un sistema. Cada clase amplía el anterior.



☐ El próximo paso es de ustedes: ¿qué proceso en su organización podría ser el primer prototipo?



Recursos para seguir

Vía No-Code

- **v0.app** — interfaces con IA, plan gratuito
- **n8n.io** — automatización open source y self-hosted
- **make.com** — automatización de procesos

Vía Código

- **claude.ai/code** — Claude Code en terminal
- **agente-viajes/** — código del día en el repo del programa
- **uv + Python 3.11** — stack recomendado

Para Prompting

- **Anthropic Prompt Library** — prompts verificados por caso de uso
- **Clase 1 del programa** — la base está ahí

Programa de Especialización IA Generativa y LLMs
para la Transformación Organizacional

Clase 3 de 3

FEN · Abril 2026

El prompt es un contrato. El agente es un colaborador. La herramienta es tuya. Ahora construye.

